Get started with Office Add-ins

Deployment of Office Add-ins

Clavin Fernandes

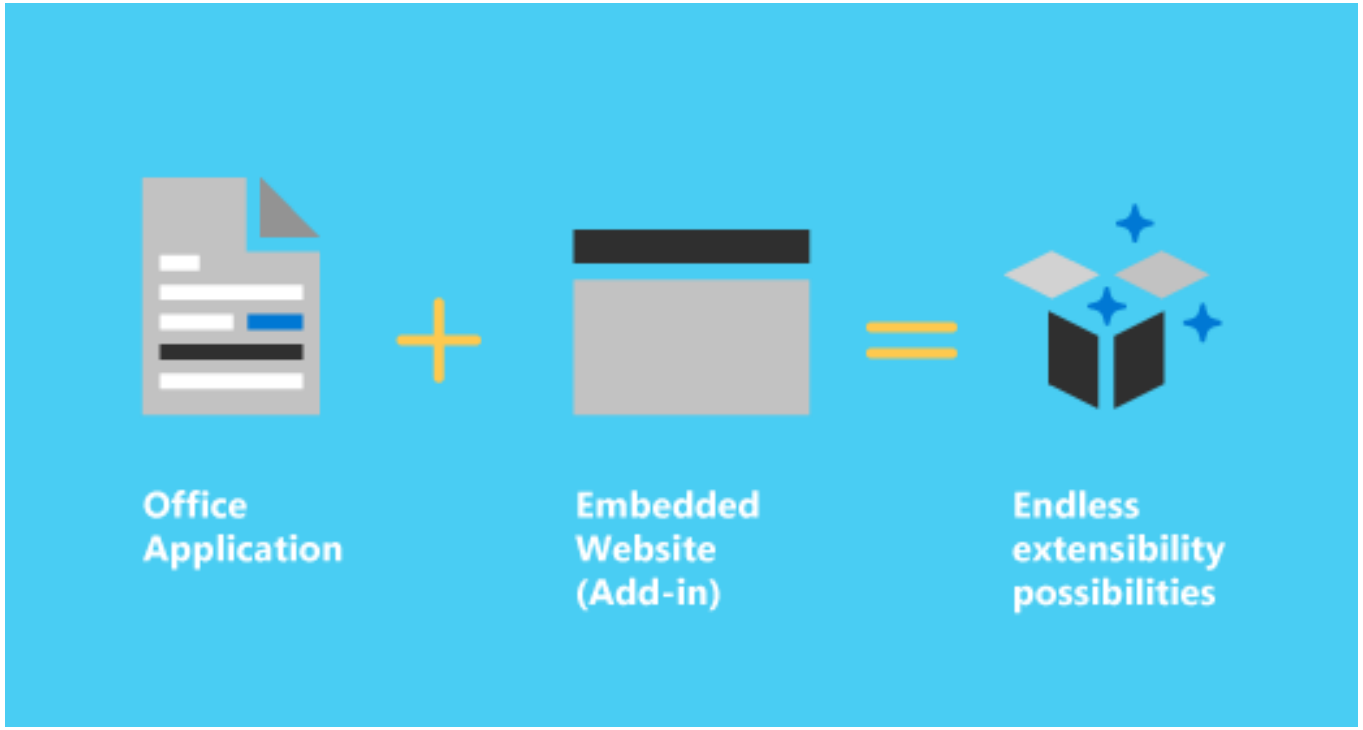
Solution Enginner @Nutrient.IO, Microsoft MVP and Leader
PPUG.org

Power Platform & AI User Group

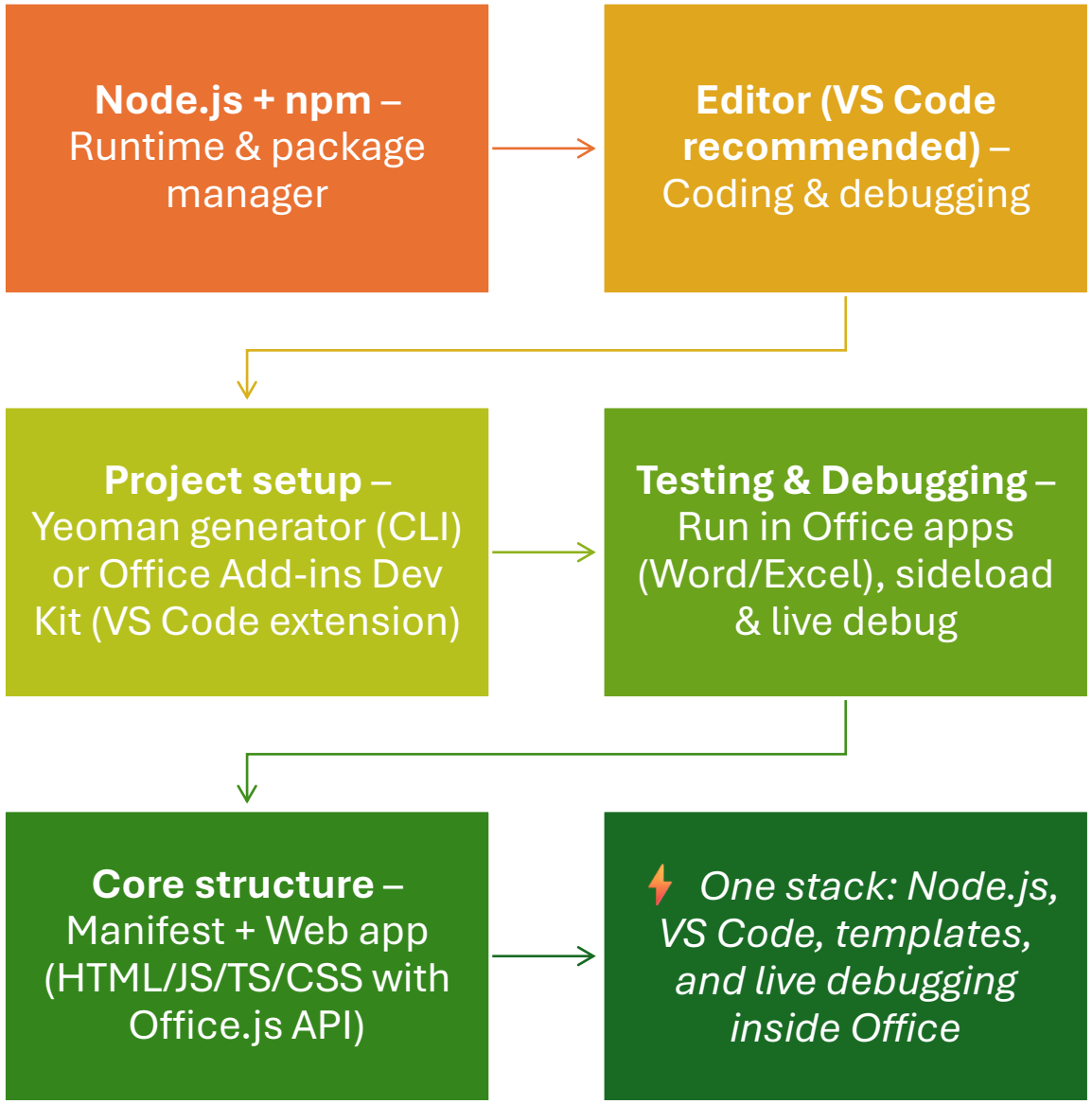
PPUG.ORG – Power Platform, AI and M365 user group.

Office Add-ins platform overview

- You can use the Office Add-ins platform to build solutions that extend Office applications and interact with content in Office documents.
- With Office Add-ins, you can use familiar web technologies such as HTML, CSS, and JavaScript to extend and interact with Outlook, Excel, Word, PowerPoint, etc.
- Your solution can run in Office across multiple platforms, including Windows, Mac, iPad, and in a browser.



Tooling for Office Add-in Development



Office Add-ins are primarily web apps

- Custom task panes or content panes designed to execute user-defined functions and commands, seamlessly hosted and managed within a web application.

Demo: Let's provision our first Microsoft Add-IN project

```
$ yo office

     _-----_     ╭──────────────────────────╮
    |       |    │    Welcome to the Office │
    |--(o)--|    │   Add-in generator, by   │
   `---------´   │ @OfficeDev! Let's create │
    ( _´U`_ )    │    a project together!   │
    /___A___\   /╰──────────────────────────╯
     |  ~  |
   __'.___.'__
 ´   `  |° ´ Y `

? Choose a project type: Office Add-in Task Pane project
? Choose a script type: JavaScript
? What do you want to name your add-in? My Office Add-in
? Which Office client application would you like to support? Word
```

- **yo office**

SPONSORED BY

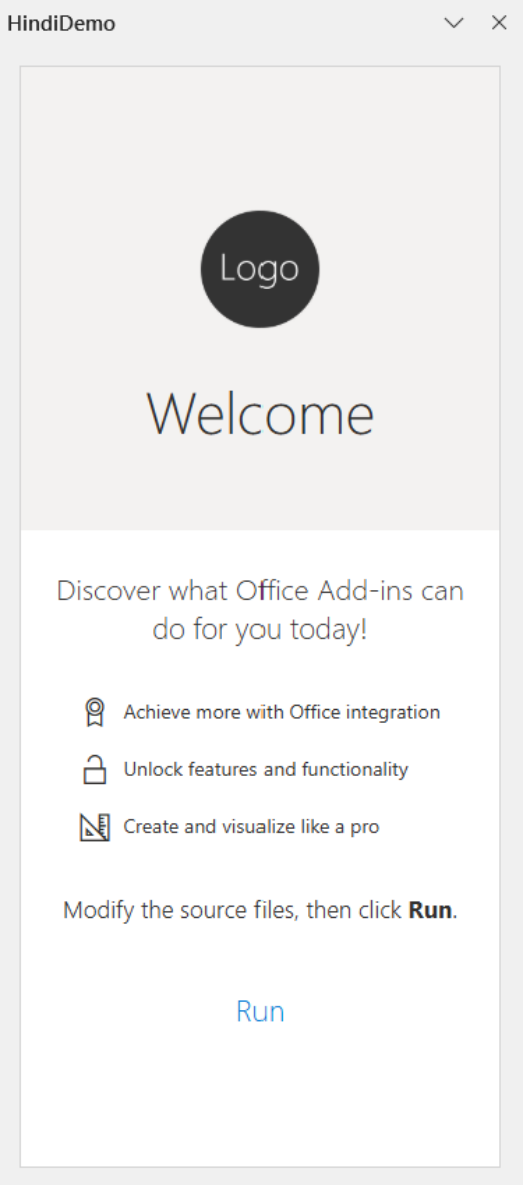
Nutrient

Understanding the project

Power Platform & AI User Group

Understanding the project

- **Understand the two parts of an Office Add-in…**
- An Office Add-in consists of two parts.
- The add-in manifest that defines the settings and capabilities of the add-in.
  - The web application that defines the UI and functionality of add-in components such as task panes, content add-ins, and dialog boxes.
  - The web application uses the Office JavaScript API to interact with content in the Office document where the add-in is running. Your add-in can also do other things that web applications typically do, like call external web services, facilitate user authentication, and more.



Add-in manifest

- **Office Add-in manifest file**
- XML file that defines settings & capabilities of the Office Add-in
- Controls how the add-in is rendered & behaves in Office apps
- Contains:
  - Metadata
  - How it integrates with Office
  - Permissions
  - Image locations (used for branding)
  - Default dimensions (content Add-ins)
  - Height & activation rules (Outlook Add-ins)

Initialize Add-in & notify Office it’s loaded

- Add-ins must notify Office they have loaded within 5 seconds of being initiated
  - Do this by implementing the Office.onReady() method
  - Failure to do this results in Office displaying an error

SPONSORED BY

Nutrient

Your First Word - AddIN, lets write some code

Power Platform & AI User Group

Office JavaScript API (Office JS)

- Enables Office Add-ins and components to interact with objects in Office client apps
- Provides two categories of JavaScript object models:
  - Common API – shared across all Office applications
  - Host-specific APIs – tailored for Word, Excel, Outlook, etc.

```
<!-- Office JavaScript API -->
<script type="text/javascript" src="https://appsforoffice.microsoft.com/lib/1/hosted/office.js"></script>

<!-- For more information on Fluent UI, visit https://developer.microsoft.com/fluentui#/. -->
<link rel="stylesheet" href="https://res-1.cdn.office.net/files/fabric-cdn-prod_20230815.002/office-ui-fabric-core/11.1.0/css/fabric.min.css"/>
```

Add-in code execution environment (Word)

- Runs in a sandboxed web runtime (no direct access to Word's process)..
- You work with proxy objects (`Document`, `Range`, `Paragraph`, `ContentControl`, etc.).
- Pattern: queue actions → load properties → `context.sync()` to round-trip with Word.
- Use `Word.run` to manage the request context and auto-dispose it.
- Feature availability is defined by Requirement Sets (e.g., `WordApi 1.3+`).
- Common objects: `document.body`, `getSelection()`, `ParagraphCollection`, `Table`, `ContentControls`.
- You can insert rich content (text, HTML, OOXML, images) and handle events (content controls, selection changes in newer sets).

Initialize your add-in & let Office know it's ready

```
// Runs when Office.js is ready and the host is known
Office.onReady((info) => {
  if (info.host === Office.HostType.Excel) {
    // your init: wire up UI, register handlers, etc.
    console.log("Add-in ready for", info.host);
  }
});

// (Optional legacy support)
Office.initialize = () => { /* no-op for older docs that still call initialize */ };
```

- Call Office.onReady() as soon as your page loads.
- Put your initialization logic inside the onReady handler.
- If the add-in doesn't initialize promptly, Office may show an Add-in Error.

Read selection (hello, proxies)

```
Word.run(async (context) => {
  const selection = context.document.getSelection();
  selection.load("text");
  await context.sync();
  console.log(`Selected text: ${selection.text}`);
})
```

Write to the document

```
Word.run(async (context) => {
  const body = context.document.body;
  body.insertParagraph("Hello from Office.js 👋 ",
Word.InsertLocation.end);
  body.insertText(" This was appended.", Word.InsertLocation.end);
  await context.sync();
});
```

Search & format results

```
Word.run(async (context) => {
  const results = context.document.body.search("Contoso", {
matchCase: false });
  results.load("text, font/color");
  await context.sync();


  results.items.forEach(r => (r.font.color = "blue"));
  await context.sync();
});
```

Create and tag a content control

```
• Word.run(async (context) => {
•   const para = context.document.body.insertParagraph("Customer
  Name", Word.InsertLocation.end);
•   const cc = para.insertContentControl();
•   cc.title = "CustomerName";
•   cc.tag = "customer_name";
•   cc.appearance = "BoundingBox";
•   await context.sync();
• });
```

Insert HTML or OOXML

```
• Word.run(async (context) => {
•   const html = "<h2>Report</h2><p><b>Status:</b> Green</p>";
•   context.document.body.insertHtml(html,
  Word.InsertLocation.end);
•   await context.sync();
• });
```

Error Handling

```
Word.run(async (context) => {
  try {
    const selection = context.document.getSelection();
    selection.load("text");
    await context.sync();
    console.log("Selected text: " + selection.text);
  } catch (error) {
    console.error("Error: " + error);
    if (error instanceof OfficeExtension.Error) {
      console.error("Debug info: " + JSON.stringify(error.debugInfo));
    }
  }
});
```

SPONSORED BY

Nutrient

Deployment of Office Add-ins

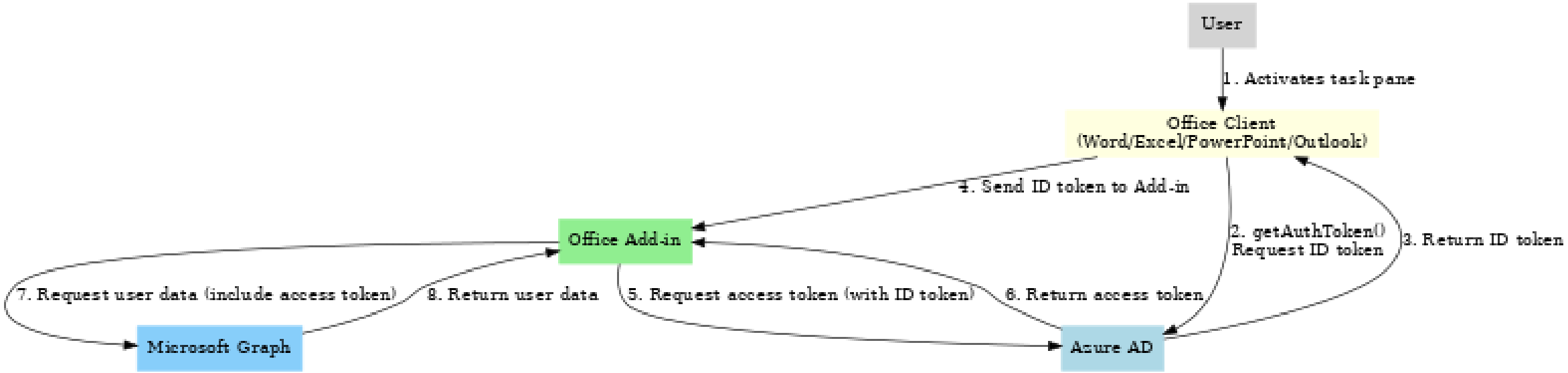
Power Platform & AI User Group

Deployment of Office Add-ins

- Multiple options for deployment of Office Add-ins
  - Sideload
  - Centralized deployment
  - AppSource
  - Manual deployment(Network share)

Deployment Options

- SSO



Examples

Deploy a single sign-on (SSO) Office Add-in to Microsoft Azure App Service

Deploy Office Add-in to M365 tenant

SPONSORED BY

Nutrient

Thank you!!!